

THE UNIVERSITY OF TULSA

Project 5: Unsupervised Learning (k-Means and EM)

CS-4103 Fundamentals of Machine Learning

In the Fundamentals of Machine Learning course, your projects require you to write and run Python code. You will implement, modify, or run one or more learning algorithms, train models with multiple datasets, test the accuracy of your models, and produce graphs and tables showing the performance of your models.

You will be given a code shell to start with, the notebook `Assignment 5`. You will use scikit-learn's `KMeans` and `GaussianMixture` classes, so you do not implement the algorithms themselves. The code shell provides the data loading, the synthetic datasets, the repeated runs and their scoring, and the plots. Your task is to build and configure each model, read the learned parameters out of the fitted models, choose the number of customer segments, run the experiments, and report on the results. Your work is the notebook cells marked `TODO` and the report. You will not need to import any libraries besides those provided. Use the code shell as a guide, but feel free to restructure the code as you prefer. You may discuss algorithms with other students or chatbots, but it is up to you to write the code and the report.

Run the notebook in Google Colab. Open it from the Assignment 5 Colab link, then save your own copy with *File > Save a copy in Drive* and work in that copy. It does not need a GPU. The same notebook is in the course repository if you prefer to run it on your own computer.

Read the Harvey items *Unsupervised Learning: k-Means & EM* and *Unsupervised Learning Readings*.

1 Overview

k-Means groups the data into k clusters, assigning each point to the nearest cluster center. EM fits a Gaussian mixture model, in which each cluster is a Gaussian with its own mean, its own variance, and its own weight, and each point belongs to every cluster with some probability. You will compare k-Means and EM on synthetic data where the true clusters are known, use both to group the customers of an online shop, where the true groups are not known, and then use k-Means to compress images to a few colors.

2 Algorithms

k-Means: `sklearn.cluster.KMeans`. Where k-Means ends up depends on its starting centers, so run several restarts from different starting centers; the model keeps the best one.

EM: `sklearn.mixture.GaussianMixture`, which is trained with EM. Give each component its own variance, set a tolerance of about 10^{-5} and a higher iteration limit than the default, so that EM runs until it converges.

Pass the seed the code shell gives you as each model's random state. You will also read the learned parameters out of each fitted model: the k-Means cluster centers, and the EM component means and standard deviations. `GaussianMixture` stores variances, not standard deviations.

3 Datasets

3.1 Synthetic Gaussian Mixtures

The code shell samples 1-dimensional datasets from S Gaussian sources. Each dataset has $S \cdot N$ points, N from each source. The source means are evenly spaced, $\Delta\mu$ apart, and each source has standard deviation σ. Every experiment uses $N = 1000$ and $\Delta\mu = 2$, and fits C clusters with k-Means and C components with EM.

1. Vary the number of sources: $S \in \{3, 5, 10\}$, $\sigma = 1$, $C = S$.
2. Vary the size of the sources: $S = 5$, $\sigma \in \{1, 2, 3\}$, $C = S$.
3. Vary the number of clusters: $S = 3$, $\sigma = 1$, $C \in \{2, 3, 6, 8\}$.

Each setting is run 50 times on newly sampled data, and the results are averaged over the runs. Since you know which source each point came from, the code shell scores each clustering with the Rand index: the fraction of all pairs of points on which the clustering agrees with the sources, either because the two points share a source and a cluster, or because they come from different sources and fall in different clusters. The code shell also computes the loss of each clustering, the sum of the squared Euclidean distances of the points from the centers of the clusters they are assigned to:

$$L = \sum_{n=1}^{S \cdot N} \sum_{k=1}^{C} r_{nk} \, \|x_n - \mu_k\|_2^2,$$

where r_{nk} is 1 when point n is assigned to cluster k and 0 otherwise. This is Equation 6.2 on page 80 of the Harvard CS 181 textbook, which is also in the *Textbooks* document in the *Reference Material* folder of the *Course Content* module on Harvey. For EM, the loss uses each point's most likely component and that component's mean. When $C = S$, the code shell also prints the learned k-Means means, and the learned EM means and standard deviations, next to the true source parameters.

3.2 Customer Segmentation

The Online Retail dataset holds every sale of a UK online gift shop from December 2010 to December 2011. The code shell loads it as one row per customer: 4,338 customers, each with the days since their last order, their number of orders, and their total spend. There are no labels, so nobody knows the true groups of customers or how many there are. Cluster the customers with k-Means and with EM on two features, recency and spend. The code shell takes the logarithm of each feature and scales it to mean 0 and standard deviation 1.

Without labels there is no Rand index. Instead, the code shell plots the silhouette score of k-Means and the BIC[1] of EM for 2 to 8 clusters. A higher silhouette score and a lower BIC are better. Choose a number of clusters for each method from these plots. The code shell then fits both models, prints the size and the median recency, orders, and spend of each segment, plots the customers colored by segment, and prints the Rand index between the two segmentations.

[1] BIC $= -2\log(\hat{L}) + \log(N)\, d$, where $\hat{L}$ is the maximum likelihood of the fitted mixture, d is its number of free parameters, and N is the number of points, here the 4,338 customers. It is the AIC, $-2\log(\hat{L}) + 2d$, with the constant 2 replaced by $\log(N)$.

3.3 Image Compression

Every pixel of a color image is a point in 3 dimensions: its red, green, and blue values. Clustering the pixels with k-Means and replacing each pixel by its cluster center redraws the image with only k colors. Choose 5 images of interest to you, such as portraits, famous paintings, landmarks, or iconic objects, and compress each one to $k \in \{3, 5, 10\}$ colors. Avoid images made only of a few solid colors, such as frames from animated shows. The code shell fits each model on a random sample of the pixels and shows the original beside the compressed versions.

4 Report & Scoring

For each experiment, give the Rand index and the loss of k-Means and of EM for every setting, as the mean and standard deviation over the 50 runs, and include the fit plots. For every setting with $C = S$, compare the learned parameters with the true ones. Describe how the Rand index and the loss of each method change with the number of sources, the size of the sources, and the number of clusters, and where k-Means and EM differ.

For the customer segmentation, include the silhouette and BIC plots, say which number of clusters each plot favors, and give the numbers you chose for k-Means and EM with your reason. Include the segment tables and plots of both methods, describe the customers in each segment in one sentence, and say where k-Means and EM split the customers differently.

Include your 5 images compressed to 3, 5, and 10 colors, and comment on the quality of each compression.

- **30pts** - Each k-Means experiment is worth 10 points.
- **30pts** - Each EM experiment is worth 10 points.
- **30pts** - Customer segmentation: 10 points for choosing the numbers of clusters, 10 points for the k-Means segments, and 10 points for the EM segments.
- **10pts** - Each compressed image is worth 2 points.

Submission

Notebook: Submit the completed notebook with every cell run and its output visible. In Colab, download it with *File > Download > Download .ipynb*. A cleared notebook will be graded as though the code did not run.

Report: Submit the report as a single PDF containing the plots, tables, and comments described above.

Where: Submit both through Harvey.